
Université Cheikh Anta Diop



Analyse d'Algorithmes

RAPPORT DE PROJET

Module Python : Tri et Recherche dans une Table 2D

Présenté par :

Nafissatou Faye

Mareme Tine

Cherif Younouss Diedhiou

Abdoul Wahab Sall

Anta Diama Kama

TABLE DES MATIÈRES

1. Introduction	4
1.1 Importance des algorithmes	4
1.2 Objectif du projet	4
2. Représentation d'une Table à Deux Dimensions	5
2.1 Structure de données	5
2.2 Convention adoptée dans le module.....	5
3. Algorithmes de Tri	6
3.1 Tri à Bulle (Bubble Sort).....	6
Principe	6
Pseudo-code	6
Complexité	6
Exemple.....	6
3.2 Tri Rapide (Quick Sort).....	6
Principe	6
Pseudo-code	6
Complexité	6
Exemple.....	7
3.3 Tri par Sélection (Selection Sort).....	7
Principe	7
Pseudo-code	7
Complexité	7
3.4 Tri par Insertion (Insertion Sort)	7
Principe	7
Pseudo-code	7
Complexité	7
3.5 Tri par Fusion (Merge Sort)	7
Principe	7
Pseudo-code	8
Complexité	8
4. Algorithmes de Recherche	9
4.1 Recherche Linéaire.....	9
Principe	9
Pseudo-code	9
Complexité	9
Avantage / Inconvénient.....	9
4.2 Recherche Binaire (Binary Search).....	9
Principe	9
Pseudo-code	9

Complexité	9
4.3 Recherche par Dichotomie (version récursive)	9
Principe	9
Pseudo-code	10
Complexité	10
5. Tests et Résultats	11
5.1 Table de test générée aléatoirement	11
5.2 Résultats des tris (colonne 0, croissant)	11
5.3 Résultats des recherches	11
6. Comparaison des Performances	12
6.1 Tableau récapitulatif des tris	12
6.2 Tableau récapitulatif des recherches	12
6.3 Analyse et recommandations	12
7. Conclusion	13

1. Introduction

1.1 Importance des algorithmes

Les algorithmes constituent le fondement de l'informatique. Un algorithme est une séquence finie et non ambiguë d'instructions permettant de résoudre un problème donné. L'étude des algorithmes de tri et de recherche est essentielle pour tout informaticien, car ces opérations sont omniprésentes dans les systèmes logiciels : bases de données, moteurs de recherche, systèmes d'exploitation, etc.

La performance d'un algorithme se mesure principalement par sa complexité temporelle (temps d'exécution en fonction de la taille des données) et sa complexité spatiale (mémoire consommée). Choisir le bon algorithme peut faire la différence entre un programme qui répond en quelques millisecondes et un autre qui prend plusieurs heures.

1.2 Objectif du projet

Ce projet a pour objectif de concevoir un module Python réutilisable nommé `tri_recherche_2d`, regroupant plusieurs algorithmes classiques de tri et de recherche appliqués à des tables à deux dimensions (listes de listes). Les objectifs spécifiques sont :

- Implémenter cinq algorithmes de tri : bulle, rapide, sélection, insertion et fusion.
- Implémenter trois algorithmes de recherche : linéaire, binaire et dichotomique (récursif).
- Appliquer ces algorithmes sur une colonne spécifique d'une table 2D.
- Documenter chaque algorithme avec son principe, sa complexité et un exemple.
- Produire un module Python bien structuré, importable et réutilisable.

2. Représentation d'une Table à Deux Dimensions

2.1 Structure de données

Une table à deux dimensions est une liste de lignes en Python. Chaque ligne est elle-même une liste de valeurs. L'accès à un élément se fait via `table[i][j]` où `i` est l'indice de la ligne et `j` l'indice de la colonne.

```
table = [  
    [3, 'Charlie', 85], # ligne 0  
    [1, 'Alice', 92], # ligne 1  
    [2, 'Bob', 78], # ligne 2  
]
```

Les opérations de tri portent sur une colonne-clé choisie par l'utilisateur (paramètre `col`). Les lignes entières sont déplacées ensemble, ce qui préserve la cohérence des données.

2.2 Convention adoptée dans le module

- `table[i][j]` : ligne `i`, colonne `j`
- `col` : indice de la colonne sur laquelle porte le tri ou la recherche
- Toutes les fonctions retournent une NOUVELLE table (la table originale n'est jamais modifiée)
- Le paramètre `croissant=True` permet de choisir l'ordre de tri

3. Algorithmes de Tri

Chaque algorithme de tri est présenté selon la même structure : principe de fonctionnement, pseudo-code, complexité temporelle et exemple d'application.

3.1 Tri à Bulle (Bubble Sort)

Principe

Le tri à bulle parcourt le tableau plusieurs fois de gauche à droite. À chaque passage, il compare deux éléments adjacents et les échange s'ils sont dans le mauvais ordre. Après chaque passage, l'élément le plus grand remonte (comme une bulle) à sa position définitive en fin de tableau. L'algorithme s'arrête dès qu'aucun échange n'est effectué.

Pseudo-code

```
Pour i de 0 à n-1 :  
    echange = Faux  
    Pour j de 0 à n-i-2 :  
        Si table[j][col] > table[j+1][col] :  
            Echanger table[j] et table[j+1]  
            echange = Vrai  
    Si echange == Faux : arrêter
```

Complexité

Algorithme	Meilleur cas	Cas moyen	Pire cas
Tri à bulle	$O(n)$	$O(n^2)$	$O(n^2)$

Exemple

Table initiale (col=0) : [3, 78, 1, 92, 2]

Après tri à bulle (croissant) : [1, 2, 3, 78, 92]

3.2 Tri Rapide (Quick Sort)

Principe

Le tri rapide utilise l'approche diviser pour régner. Il choisit un élément pivot (ici le dernier élément), partitionne le tableau en deux sous-listes (éléments inférieurs et supérieurs au pivot), puis applique récursivement le même procédé sur chaque sous-liste. La liste finale est la concaténation des trois parties.

Pseudo-code

```
Fonction tri_rapide(t, col) :  
    Si longueur(t) <= 1 : retourner t  
    pivot = t[-1]  
    inferieurs = [x pour x dans t[:-1] si x[col] <= pivot[col]]  
    superieurs = [x pour x dans t[:-1] si x[col] > pivot[col]]  
    retourner tri_rapide(inferieurs) + [pivot] + tri_rapide(superieurs)
```

Complexité

Algorithme	Meilleur cas	Cas moyen	Pire cas
Tri rapide	$O(n \log n)$	$O(n \log n)$	$O(n^2)$

Exemple

Pivot = 2 → inférieurs=[1], pivot=[2], supérieurs=[3, 78, 92]

Résultat : [1, 2, 3, 78, 92]

3.3 Tri par Sélection (Selection Sort)

Principe

Le tri par sélection divise le tableau en une partie triée (à gauche) et une partie non triée (à droite). À chaque itération, il sélectionne le minimum de la partie non triée et le place au début de cette zone. La partie triée grandit d'un élément à chaque étape.

Pseudo-code

```
Pour i de 0 à n-1 :  
    idx_min = i  
    Pour j de i+1 à n-1 :  
        Si table[j][col] < table[idx_min][col] :  
            idx_min = j  
    Echanger table[i] et table[idx_min]
```

Complexité

Algorithme	Meilleur cas	Cas moyen	Pire cas
Tri par sélection	$O(n^2)$	$O(n^2)$	$O(n^2)$

3.4 Tri par Insertion (Insertion Sort)

Principe

Similaire au tri de cartes à jouer dans la main. On considère que le premier élément est déjà trié. Pour chaque nouvel élément, on le retire et on le réinsère à la bonne position dans la partie déjà triée, en décalant vers la droite les éléments plus grands que lui.

Pseudo-code

```
Pour i de 1 à n-1 :  
    ligne_courante = table[i]  
    j = i - 1  
    Tant que j >= 0 et table[j][col] > ligne_courante[col] :  
        table[j+1] = table[j]  
        j = j - 1  
    table[j+1] = ligne_courante
```

Complexité

Algorithme	Meilleur cas	Cas moyen	Pire cas
Tri par insertion	$O(n)$	$O(n^2)$	$O(n^2)$

3.5 Tri par Fusion (Merge Sort)

Principe

Algorithme diviser pour régner en trois phases : (1) DIVISER : couper récursivement le tableau en deux moitiés jusqu'à obtenir des sous-tableaux de taille 1. (2) CONQUÉRIR : un tableau de taille 1 est trivialement trié. (3) FUSIONNER : regrouper deux sous-tableaux triés en un seul tableau trié en comparant leurs éléments un à un.

Pseudo-code

```
Fonction fusion(gauche, droite, col) :  
    resultat = []  
    Tant que gauche et droite non vides :  
        Si gauche[0][col] <= droite[0][col] :  
            ajouter gauche[0] à resultat, avancer gauche  
        Sinon : ajouter droite[0] à resultat, avancer droite  
    Ajouter les restes
```

Complexité

Algorithme	Meilleur cas	Cas moyen	Pire cas
Tri par fusion	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

4. Algorithmes de Recherche

4.1 Recherche Linéaire

Principe

La recherche linéaire parcourt chaque ligne du tableau de la première à la dernière et compare la valeur de la colonne-clé avec la valeur recherchée. Dès qu'une correspondance est trouvée, elle retourne l'index de la ligne. Si aucune ligne ne correspond, elle retourne -1. Cet algorithme fonctionne sur toute table, triée ou non.

Pseudo-code

```
Pour i de 0 à n-1 :  
    Si table[i][col] == valeur : retourner i  
Retourner -1
```

Complexité

Algorithme	Meilleur cas	Cas moyen	Pire cas
Recherche linéaire	$O(1)$	$O(n)$	$O(n)$

Avantage / Inconvénient

- Avantage : fonctionne sur une table non triée.
- Inconvénient : lent sur de grandes tables (examine toutes les lignes dans le pire cas).

4.2 Recherche Binaire (Binary Search)

Principe

La recherche binaire fonctionne sur une table DÉJÀ TRIÉE en ordre croissant. Elle maintient deux bornes (gauche et droite). À chaque itération, elle calcule le milieu et compare la valeur centrale : si égale → trouvé ; si inférieure → chercher dans la moitié droite ; si supérieure → chercher dans la moitié gauche. L'espace de recherche est réduit de moitié à chaque étape.

Pseudo-code

```
gauche = 0 ; droite = n - 1  
Tant que gauche <= droite :  
    milieu = (gauche + droite) // 2  
    Si table[milieu][col] == valeur : retourner milieu  
    Sinon si table[milieu][col] < valeur : gauche = milieu + 1  
    Sinon : droite = milieu - 1  
Retourner -1
```

Complexité

Algorithme	Meilleur cas	Cas moyen	Pire cas
Recherche binaire	$O(1)$	$O(\log n)$	$O(\log n)$

4.3 Recherche par Dichotomie (version récursive)

Principe

La recherche par dichotomie applique le même principe que la recherche binaire mais de façon récursive. On divise l'espace de recherche en deux à chaque appel récursif. Le cas de base est atteint quand la zone est vide (non trouvé) ou quand la valeur centrale est égale à la valeur recherchée (trouvé).

Pseudo-code

```
Fonction dichotomie(table, col, valeur, gauche, droite) :  
    Si gauche > droite : retourner -1  
    milieu = (gauche + droite) // 2  
    Si table[milieu][col] == valeur : retourner milieu  
    Sinon si table[milieu][col] < valeur :  
        retourner dichotomie(table, col, valeur, milieu+1, droite)  
    Sinon :  
        retourner dichotomie(table, col, valeur, gauche, milieu-1)
```

Complexité

Algorithme	Meilleur cas	Cas moyen	Pire cas
Dichotomie (récursif)	$O(1)$	$O(\log n)$	$O(\log n)$

5. Tests et Résultats

5.1 Table de test générée aléatoirement

Pour les tests, une table de 8 lignes et 3 colonnes a été générée avec la fonction `generer_table_aleatoire(graine=42)`. La graine fixe garantit des résultats reproductibles.

Table générée (graine=42) :

```
[ 0] [68, 4, 71]
[ 1] [20, 88, 52]
[ 2] [15, 79, 33]
[ 3] [98, 32, 64]
[ 4] [55, 11, 95]
[ 5] [37, 91, 20]
[ 6] [79, 44, 57]
[ 7] [60, 83, 68]
```

5.2 Résultats des tris (colonne 0, croissant)

Résultat (tous les tris donnent le même ordre) :

```
[ 0] [15, 79, 33]
[ 1] [20, 88, 52]
[ 2] [37, 91, 20]
[ 3] [55, 11, 95]
[ 4] [60, 83, 68]
[ 5] [68, 4, 71]
[ 6] [79, 44, 57]
[ 7] [98, 32, 64]
```

5.3 Résultats des recherches

```
[Recherche linéaire] valeur=98 → index trouvé : 3
[Recherche linéaire] valeur=9999 → index trouvé : -1 (non trouvé)

[Recherche binaire]  valeur=98 → index trouvé : 7 (table triée)
[Recherche binaire]  valeur=9999 → index trouvé : -1 (non trouvé)

[Recherche dichotomie] valeur=98 → index trouvé : 7 (table triée)
[Recherche dichotomie] valeur=9999 → index trouvé : -1 (non trouvé)
```

6. Comparaison des Performances

6.1 Tableau récapitulatif des tris

Algorithme	Meilleur cas	Cas moyen	Pire cas
Tri à bulle	$O(n)$	$O(n^2)$	$O(n^2)$
Tri rapide	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Tri par sélection	$O(n^2)$	$O(n^2)$	$O(n^2)$
Tri par insertion	$O(n)$	$O(n^2)$	$O(n^2)$
Tri par fusion	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

6.2 Tableau récapitulatif des recherches

Algorithme	Meilleur cas	Cas moyen	Pire cas
Recherche linéaire	$O(1)$	$O(n)$	$O(n)$
Recherche binaire	$O(1)$	$O(\log n)$	$O(\log n)$
Dichotomie (récursif)	$O(1)$	$O(\log n)$	$O(\log n)$

6.3 Analyse et recommandations

Algorithmes de tri :

- Le tri par fusion est le plus stable : $O(n \log n)$ garanti dans tous les cas.
- Le tri rapide est le plus rapide en pratique ($O(n \log n)$ en moyenne), mais peut dégénérer à $O(n^2)$.
- Le tri à bulle et le tri par insertion sont efficaces sur des données quasi-triées ($O(n)$).
- Le tri par sélection effectue toujours $O(n^2)$ comparaisons, quelle que soit l'entrée.

Algorithmes de recherche :

- La recherche binaire et la dichotomie sont $O(\log n)$: BEAUCOUP plus rapides que la linéaire sur de grands tableaux.
- La recherche linéaire ne nécessite pas de tri préalable : avantage si le tableau n'est pas trié.
- La dichotomie récursive est élégante mais consomme de la pile d'appels (risque de stack overflow sur de très grands tableaux).

7. Conclusion

Ce projet a permis de concevoir et d'implémenter un module Python complet regroupant cinq algorithmes de tri et trois algorithmes de recherche, appliqués à des tables à deux dimensions. Chaque algorithme a été documenté avec son principe, son pseudo-code, sa complexité et un exemple concret.

L'analyse comparative révèle que le choix de l'algorithme dépend fortement du contexte : taille des données, état initial du tableau, contraintes mémoire et exigences de performance. Le tri par fusion offre les meilleures garanties théoriques, tandis que la recherche binaire surpasse largement la recherche linéaire sur les grandes tables triées.

Ce travail renforce notre compréhension des structures de données et de l'analyse de complexité, compétences fondamentales pour tout développeur. Le module produit est réutilisable, bien documenté et disponible sur GitHub pour consultation et utilisation.